



Torneo Chileno de Programación 2026

03-10-2026

Universidad de Tarapacá, Arica

Universidad Católica del Norte, Coquimbo

Universidad del Viña del Mar, Viña del Mar

Pontificia Universidad Católica de Valparaíso, Valparaíso

Universidad Diego Portales, Santiago

Universidad Técnica Federico Santa María, Santiago

Universidad Católica del Maule, Talca

Universidad Católica de la Santísima Concepción, Concepción

Universidad Austral de Chile, Valdivia

Agradecimientos

De parte de la Sociedad Chilena de Programación Competitiva, queremos agradecer a todas las personas que dedicaron su tiempo y esfuerzo a trabajar en este problem set: **Camilo Castro, Javier Oliva, Marcelo Lemus, Vicente Villarroel, Gabriel Carmona Diaz¹, Blaz Korecic y Sebastián Torrealba**. También queremos agradecer a quienes nos ayudaron a probar los problemas: **Martin Muñoz y Carlos Lagos**.

Gracias a todos ellos, por tercer año consecutivo tenemos un *Torneo Chileno de Programación*, hecho por chilenos y para chilenos.

Agradecemos también a *ICPC Chile* por organizar esta competencia y hacer posible que se realice en distintas sedes a lo largo del país.

Finalmente, les deseamos mucho éxito a todas y todos los competidores. ¡Esperamos que disfruten la competencia y, por supuesto, que se diviertan resolviendo los problemas!

Gabriel Carmona Tabja², organizador del problem set TCP-2026

Sociedad Chilena de Programación Competitiva.

P.D.: Personalmente quiero agradecer especialmente a los autores de los problemas por aceptar que todos los enunciados tuvieran temática de Super Mario.

¹Había que diferenciar entre los dos Gabriel Carmona de Programación Competitiva en Chile

²Otro Gabriel Carmona

A. Al derecho y al revés

Límite de tiempo: 1 segundo

Límite de memoria: 256 megabytes

Mario está en un nivel donde la gravedad se invierte constantemente, así que a veces lee los números de pie y a veces de cabeza. Por eso le interesan los números que se ven igual desde ambos lados.

Existen algunos dígitos que al rotarlos 180 grados quedan exactamente igual, estos dígitos son:

1, 2, 5, 8

Al rotar 180 grados un número completo ocurren dos cosas: se rota cada uno de sus dígitos y, además, se invierte el orden en que aparecen. Al usar otros números no se cumple esta condición, como por ejemplo:

347 $\xrightarrow{180^\circ}$ 743

Figura 1: Este número al rotarlo no tiene sentido (Mario no habla portugués)

Decimos que un número se lee **al derecho y al revés** si al rotarlo 180 grados se obtiene exactamente el mismo número. Por ejemplo, 1221 se lee al derecho y al revés, pero 1281 no: al rotarlo se lee 1821.

1281 $\xrightarrow{180^\circ}$ 1821 1221 $\xrightarrow{180^\circ}$ 1221

Figura 2: Este número no se lee al derecho y al revés Figura 3: Este número sí se lee al derecho y al revés

Mario está a punto de terminar el nivel, pero encuentra que la salida está cerrada. Para abrir la puerta, hay un lugar para poner un número de n dígitos, y un cartel que dice:

divisible por tres

¿Puedes entregarle a Mario un número que se lea al derecho y al revés y que cumpla con esta condición?

Entrada

La única línea contiene un entero n ($1 \leq n \leq 10^5$), la cantidad de dígitos que debe tener el número.

Salida

En la primera línea imprime **SI** en caso de que exista dicho número, o **NO** en caso de que no exista.

Si existe, en la segunda línea imprime el número de n dígitos **que sea divisible por 3**. Puede haber más de una respuesta válida; se acepta cualquiera de ellas.

Ejemplos

Entrada 1	Salida 1
2	NO
Entrada 2	Salida 2
4	SI 1221
Entrada 3	Salida 3
6	SI 111111

B. Bowser's Minions

Límite de tiempo: 2 segundos

Límite de memoria: 256 megabytes

Tras ser derrotado por Mario nuevamente, Bowser se dió cuenta de que necesitaba cambiar de estrategia. Por ello, trasladó su castillo del Reino Champiñón al Reino Árbol.

A diferencia del Reino Champiñón, donde todos los niveles forman un segmento contiguo, los niveles del reino árbol tienen la estructura de un árbol. Un árbol es un grafo no-dirigido $G = (V, E)$ que no tiene ciclos y del que se puede llegar desde un nodo a cualquier otro.

Bowser ordenó a sus secuaces para que se posicionen en los nodos del árbol, para poder atacar a Mario. Para su sorpresa, muchos de ellos decidieron el mismo nodo. Esto representa un gran problema porque reduce drásticamente la eficacia del ataque.

Para remediar esta situación, Bowser decidió ordenar a sus secuaces que se movieran. En concreto, realizó el siguiente procedimiento secuencialmente:

Mientras exista un nodo v que tenga más de un vecino y más de un secuaz:

Elegir dos secuaces ubicados en v

Mover cada uno de ellos a un vecino distinto de v

Kamek tiene dudas sobre la validez de este procedimiento. En particular, quiere saber:

- Si el proceso termina independientemente de las decisiones tomadas.
- Si la configuración final es única, es decir, si todas las ejecuciones que terminan producen la misma distribución de secuaces.

¿Puedes ayudarlo?

Entrada

La primera entrada contiene un entero n , el número de nodos del árbol ($1 \leq n \leq 500$).

La segunda línea contiene n enteros $a_1, a_2, \dots, a_n$, donde a_i indica la cantidad inicial de secuaces ubicados en el nodo i ($0 \leq \sum_{i=1}^n a_i \leq n$, y $0 \leq a_i$).

Luego, siguen $n - 1$ líneas. Cada una contiene dos enteros u, v , que indican que existe una arista entre los nodos u y v ($1 \leq u, v \leq n$).

Salida

La salida debe contener dos líneas.

En la primera línea, se debe imprimir:

- **termina**, si toda ejecución posible del proceso termina;
- **puede no terminar**, si existe alguna ejecución que no termina.

En la segunda línea, se debe imprimir:

- **unico**, si termina y todas las ejecuciones producen la misma configuración final;
- **no unico**, en caso contrario.

Ejemplos

Entrada 1	Salida 1
8 0 1 1 1 1 1 1 1 1 2 1 3 1 4 1 5 1 6 1 7 1 8	termina unico

C. Cumpleaños masivo

Límite de tiempo: 1 segundo

Límite de memoria: 256 megabytes

Peach tiene muchos, muchísimos amigos, los Toads. Tantos, que si celebraran cada cumpleaños por separado habría fiesta casi todos los días del año. Para no morir en el intento, Peach y sus muchos amigos decidieron celebrarlos todos juntos en un solo *cumpleaños masivo*. Hay un problema: con tanta gente, nunca hay suficiente torta.

Peach ya horneó n bizcochos, y el i -ésimo tiene tamaño a_i . Como las recetas se repitieron bastante, varios bizcochos pueden tener el mismo tamaño.

Una torta como corresponde tiene exactamente tres pisos, y cada piso tiene que ser de un tamaño distinto. Dos pisos del mismo tamaño no forman una torta sino un ladrillo, y nadie quiere cantar el cumpleaños a un ladrillo. Formalmente, una *torta* es un conjunto de exactamente tres bizcochos con tamaños distintos entre sí, y cada bizcocho puede usarse en a lo más una torta.

Para que alcance para todos, ¿cuál es la máxima cantidad de tortas que se pueden armar?

Entrada

La primera línea contiene un entero n ($1 \leq n \leq 2 \cdot 10^5$), la cantidad de bizcochos.

La segunda línea contiene n enteros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$), los tamaños de los bizcochos.

Salida

Imprime un único entero: la máxima cantidad de tortas que se pueden armar.

Ejemplos

Entrada 1	Salida 1
6 1 2 3 1 2 3	2
Entrada 2	Salida 2
7 4 4 4 4 7 7 9	1
Entrada 3	Salida 3
5 10 10 10 10 10	0

Nota

En el primer ejemplo, se pueden armar dos tortas, cada una con pisos de tamaños 1, 2 y 3.

En el segundo ejemplo, se puede armar una torta con pisos de tamaños 4, 7 y 9. Para armar dos tortas se necesitarían 6 bizcochos, pero cada tamaño puede aparecer a lo más una vez por torta: se pueden aprovechar solo dos bizcochos de tamaño 4, los dos de tamaño 7 y el único de tamaño 9, que suman 5.

En el tercer ejemplo, todos los bizcochos tienen el mismo tamaño, así que solo se pueden armar ladrillos. No se puede armar ninguna torta.

D. Desierto Kalimari

Límite de tiempo: 5 segundos

Límite de memoria: 256 megabytes

Bowser y sus secuaces llevaban días y días sin recibir visitas en su castillo, por lo que decidieron irse de vacaciones al Desierto Kalimari a competir en carreras de autos. Como era de esperarse en un desierto, no tenían dónde competir, por lo que decidieron traer ellos mismos los materiales necesarios para construir su propia pista de carreras.

Desafortunadamente, minutos antes de comenzar a montar la pista, se dieron cuenta de que las cajas con los materiales no estaban: ¡Bowser Jr. las había olvidado en una de las mesetas más ventosas de la zona!

La cima de la meseta puede representarse como un polígono convexo P de n vértices, y cada caja puede representarse como un punto dentro de P o sobre su frontera. Por suerte, en la estación meteorológica del desierto saben exactamente cómo soplará el viento.

El viento soplará t veces. La i -ésima ráfaga está representada por un vector $w_i \in \mathbb{R}^2$. Si una caja se encuentra en la posición x inmediatamente antes de la i -ésima ráfaga, esta será desplazada hasta $x + w_i$.

Si después de alguna ráfaga, una caja queda estrictamente fuera de P , caerá de la meseta y se perderá. Las cajas que caigan dejarán de verse afectadas por las ráfagas posteriores.

Bowser y sus secuaces quieren saber si vale la pena regresar a buscar las cajas o si sería mejor devolverse a su castillo a buscar más materiales (y asegurarse de que ningún Mario haya entrado sin permiso).

Determina cuántas cajas permanecen sobre la meseta después de las t ráfagas de viento.

Entrada

La primera línea de entrada contiene 3 enteros positivos n, q, t , separados por espacios: n , el número de vértices del polígono; q , el número de cajas; y t , el número de ráfagas de viento. ($3 \leq n \leq 10^5, 1 \leq q \leq 10^5, 1 \leq t \leq 10^5$)

Las siguientes n líneas contienen 2 enteros x_i, y_i , separados por espacios: las coordenadas del i -ésimo vértice del polígono P . Los vértices se entregan en orden antihorario. ($-10^9 \leq x_i, y_i \leq 10^9$)

Las siguientes q líneas contienen 2 enteros x_i, y_i , separados por espacios: las coordenadas de la i -ésima caja. Se garantiza que cada caja se encuentra dentro de P o sobre su frontera. ($-10^9 \leq x_i, y_i \leq 10^9$)

Las siguientes t líneas contienen 2 enteros a_i, b_i , separados por espacios, que representan el vector $w_i = (a_i, b_i)$ de la i -ésima ráfaga de viento. ($-10^9 \leq a_i, b_i \leq 10^9$)

Se garantiza que P no contiene tres puntos colineales. Además, para cualquier caja, desplazar su posición inicial una distancia de a lo más 10^{-4} no cambia la respuesta.

Salida

Imprime un único entero: el número de cajas que permanecen sobre la meseta después de las t ráfagas de viento.

Ejemplos

Entrada 1	Salida 1
4 4 2 0 0 5 0 5 5 0 5 1 1 4 1 2 4 2 2 2 0 0 2	2

Nota

En el ejemplo, la meseta es el cuadrado de vértices $(0,0)$, $(5,0)$, $(5,5)$ y $(0,5)$. El viento sopla primero hacia la derecha, $w_1 = (2,0)$, y luego hacia arriba, $w_2 = (0,2)$.

- La caja en $(1,1)$ pasa por $(3,1)$ y termina en $(3,3)$: permanece sobre la meseta.
- La caja en $(4,1)$ es llevada a $(6,1)$ por la primera ráfaga, queda fuera de P y cae.
- La caja en $(2,4)$ pasa por $(4,4)$, pero la segunda ráfaga la lleva a $(4,6)$ y cae.
- La caja en $(2,2)$ pasa por $(4,2)$ y termina en $(4,4)$: permanece sobre la meseta.

Por lo tanto, quedan 2 cajas sobre la meseta.

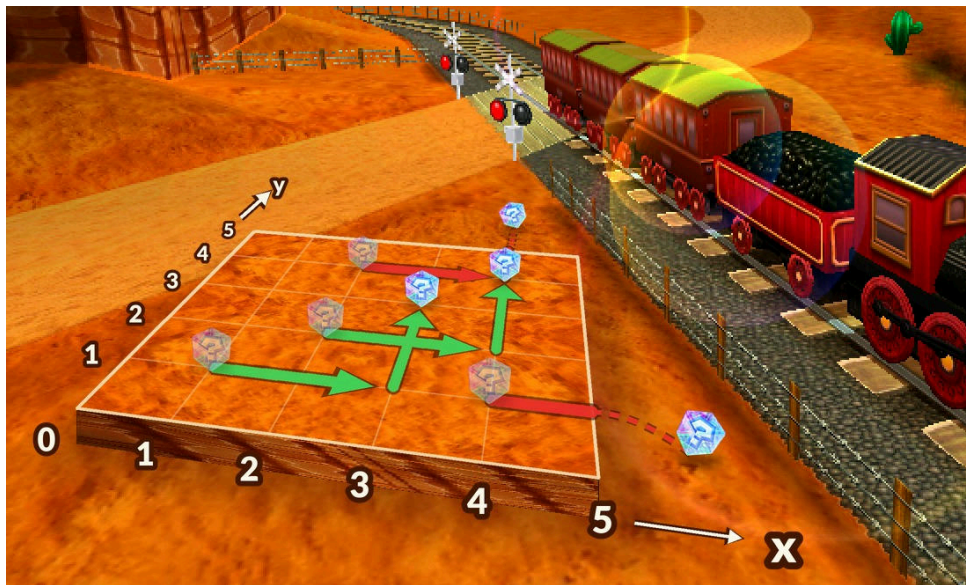


Figura 4: Trayectoria de cada caja en el ejemplo. Las cajas translúcidas marcan la posición inicial; las trayectorias verdes terminan sobre la meseta y las rojas caen.

E. Exploding Bittens

Límite de tiempo: 1 segundo

Límite de memoria: 256 megabytes

Mario y sus amigos se reunieron para jugar un novedoso juego que le habían regalado en el Campo de los Bob-omb: Exploding Bittens.

Exploding Bittens se juega con un mazo de n cartas donde cada una tiene uno de los siguientes dos tipos:

- **Carta Bob-omb:** el jugador que la saque explota y es eliminado de la partida
- **Carta estrella:** no ocurre nada y el jugador permanece en la partida.

Hay p jugadores, numerados del 1 al p , que juegan por turnos en orden cíclico. En su turno, el jugador actual saca la carta que se encuentra en la parte superior del mazo. Los jugadores eliminados son ignorados en los turnos siguientes.

La partida termina cuando no quedan cartas en el mazo. Todos los jugadores que sigan con vida en ese momento son considerados ganadores.

Luigi está desesperado por ganar y te ofrece una vida extra a cambio de reordenar el mazo antes de que comience la partida. Sabes que el mazo contiene exactamente m cartas Bob-omb y que Luigi ocupa la posición k en el orden de los jugadores.

Determina si es posible reordenar las cartas del mazo de manera que Luigi sobreviva hasta el final de la partida. Si es posible, imprime cualquier orden del mazo que lo consiga.

Entrada

La primera línea de entrada contiene 4 enteros n, m, k, p , separados por espacios: n , el número total de cartas; m , el número de cartas Bob-omb en el mazo; k , la posición de Luigi en el orden de juego; y p , el número de jugadores. ($1 \leq n \leq 100, 0 \leq m \leq n, 1 \leq k \leq p \leq n$)

Salida

Si no existe una forma de reordenar el mazo para que Luigi sobreviva, imprime **NO** en la primera línea.

Si existe, imprime **SI** en la primera línea y, en la segunda línea, un string de longitud n que represente el orden del mazo. El i -ésimo carácter debe ser **B** si la i -ésima carta es una carta Bob-omb, o **E** si es una carta estrella.

Puede haber más de una respuesta válida; se acepta cualquiera de ellas.

Ejemplos

Entrada 1	Salida 1
4 1 2 2	SI BEEE
Entrada 2	Salida 2
6 6 1 3	NO

Nota

En el primer caso de ejemplo, el mazo inicialmente contiene las cartas "BEEE". El primer jugador saca la carta Bob-omb y queda eliminado. El único jugador restante es Luigi, que sacará cartas estrella hasta que el mazo quede vacío.

En el segundo caso de ejemplo, todas las cartas son cartas Bob-omb. Luigi juega primero, así que sin importar el orden en su primer turno sacará una y quedará eliminado.

F. Formación comprimida de soldados

Límite de tiempo: 2 segundos

Límite de memoria: 256 megabytes

Mario es el general del ejército del Reino Champiñón. El reino está a punto de enfrentar una guerra muy brutal, y Mario ha formado a todo su ejército en una única línea de ataque.

Como el ejército es enorme, la formación no se anota soldado por soldado. En su lugar se registran solo los bloques: tramos consecutivos de soldados del mismo tipo, indicando el tipo y cuántos soldados hay en el tramo.

Por ejemplo, si la formación completa es “TTTTYYKKK” la anotación guardada sería “(T, 4) (Y, 2) (K, 3)”

es decir: 4 Toads seguidos, luego 2 Yoshis seguidos y al final 3 Kongs seguidos.

Luigi (subgeneral) debe anunciar en voz alta qué tipo de soldado ocupa cierta posición de la línea. Como la anotación comprimida sigue siendo grande, necesita que lo resolvamos rápido.

Entrada

La primera línea contiene un entero n ($1 \leq n \leq 10^5$): la cantidad de bloques anotados.

Las siguientes n líneas describen la formación de izquierda a derecha. La línea i contiene un carácter t_i y un entero c_i ($1 \leq c_i \leq 10^9$), separados por un espacio, donde t_i es el tipo de soldado y c_i la cantidad de soldados del bloque. El carácter t_i puede ser cualquier letra del alfabeto inglés, mayúscula o minúscula (las mayúsculas y minúsculas se consideran tipos distintos). Dos bloques consecutivos pueden tener el mismo tipo.

Se garantiza que $\sum_{i=1}^n c_i \leq 10^9$. Sea $L = \sum_{i=1}^n c_i$ el largo total de la formación.

La siguiente línea contiene un entero q ($1 \leq q \leq 10^5$): la cantidad de órdenes que debe anunciar Luigi.

Cada una de las siguientes q líneas contiene un entero p_j ($1 \leq p_j \leq L$): la posición del soldado consultado, contando desde el inicio de la línea y empezando en 1.

Salida

Imprime q líneas. En la j -ésima línea, el carácter correspondiente al tipo del soldado que ocupa la posición p_j de la formación.

Ejemplos

Entrada 1	Salida 1
3	T
T 4	Y
Y 2	Y
K 3	K
5	K
1	
5	
6	
7	
9	

Nota

La formación del ejemplo es “TTTTYYKKK”. Las posiciones 1 y 5 caen en el primer y segundo bloque respectivamente; la posición 9 es el último Kong.

G. Ganar en 145

Límite de tiempo: 2 segundos

Límite de memoria: 256 megabytes

Este problema es interactivo

Luigi recorre un nivel con n bloques en fila. Cada bloque i esconde un valor a_i , y los valores están ordenados de forma estrictamente creciente. Luigi puede golpear un bloque para descubrir su valor, pero, como en todo nivel de Luigi, solo puede avanzar hacia la derecha.

Las consultas tienen dos restricciones:

- Cada consulta debe ser a una posición estrictamente mayor que la de la consulta anterior.
- Luigi tiene una única vida extra: puede reiniciar el nivel una sola vez. Al reiniciar, la siguiente consulta puede ser a cualquier posición, y desde ahí vuelve a regir la restricción anterior.

Dado un valor x que aparece en algún bloque, ayuda a Luigi a encontrar su posición usando a lo más 145 consultas.

Interacción

Tu programa se comunica con el juez imprimiendo una instrucción por línea:

- `? i` consulta la posición i ($1 \leq i \leq n$). El juez responde con a_i en una línea.
- `R` reinicia. El juez no responde nada. No cuenta como consulta.
- `! p` informa que x está en la posición p . No cuenta como consulta, y tu programa debe terminar inmediatamente después de imprimirla.

Recibirás veredicto incorrecto si superas las 145 consultas, si reinicias dos veces, si consultas una posición que no respeta las restricciones o si la posición informada es equivocada.

El arreglo $A = \{a_1, \dots, a_n\}$ se fija antes de que tu programa comience y no cambia durante la interacción. Después de imprimir cada línea, recuerda vaciar el búfer de salida.

Entrada

La primera línea contiene dos enteros n y x ($1 \leq n \leq 5000$, $1 \leq x \leq 10^9$), el tamaño del arreglo y el valor que debes localizar. Se garantiza que x aparece en a y que $1 \leq a_i \leq 10^9$.

Salida

Cuando determines la posición, imprime `! p`, donde p es la posición en la que se encuentra x . Como el arreglo es estrictamente creciente, esta posición es única.

Ejemplos

Entrada 1	Salida 1
4 5	
2	? 1
9	? 3
5	R
	? 2
	! 2

Nota

- Los saltos de línea son para mejorar la legibilidad; no están incluidos en la interacción del juez.

Este problema es *interactivo*, por lo que en algunos momentos es importante que imprimas antes de continuar leyendo. Algunos lenguajes no imprimen inmediatamente al realizar una operación de impresión, por lo que es necesario realizar una operación de *flush* manualmente:

- Python: Agregar el parámetro `flush` al imprimir: `print(..., flush=True)`
- C++: Utilizar `std::cout << std::flush`; tras imprimir.
- C: Utilizar `fflush(stdout)`; tras imprimir.
- Otros lenguajes: Referir a la documentación.

H. El hadino y su regalo

Límite de tiempo: 5 segundos

Límite de memoria: 256 megabytes

Iurgi, un joven hadino del reino de las hadas ha adquirido un hermoso regalo para la boda de su hermana y lo guardó en una bodega mágica que tiene en su casa. Sin embargo ha perdido sus apuntes y no sabe como encontrar la puerta que tiene el regalo. Como la boda es mañana necesita encontrar el regalo con urgencia, es por esto que ha solicitado la estupenda ayuda de ustedes, los expertos mejor capacitados para resolver este asunto (Mario hizo esta recomendación).

Iurgi como es muy precavido ha ocultado el contenido de cada puerta, según una técnica usada por su amigo Julius. La técnica consiste en desplazar cada letra del alfabeto un número fijo de posiciones. Por ejemplo, con un desplazamiento (rotación) de 3:

A → D

B → E

C → F

...

X → A

Y → B

Z → C

HOLA → KROD

El alfabeto es rotacional o circular, es decir, cuando se llega al final del alfabeto, se vuelve a empezar desde A.

Iurgi ha puesto regalos falsos con tal de confundir algún ladrón. La misión es encontrar la puerta que contiene la palabra “regalo” (o su anagrama³), solo una puerta contiene la palabra correcta, las demás tendrán variantes como “regilo” o “rugalo”. Iurgi solamente necesita saber el índice de la puerta correcta para lograr abrirla.

Entrada

La primera línea contiene un n ($1 \leq n \leq 10^6$), donde n corresponde a la cantidad total de puertas.

La segunda línea contiene una lista con la cantidad de rotaciones con la cual se ha ocultado el posible regalo en cada puerta. Es decir, se encuentran n valores r_i ($0 \leq r_i \leq 25$) que corresponde a la rotación.

La tercera línea contiene una lista con el contenido de cada puerta. Es decir, se encuentran n palabras p_i que corresponde al contenido de cada puerta. Cada palabra es de largo exactamente 6 y solamente se utiliza las letras del ASCII inglés.

Salida

La salida debe contener un único entero que corresponda al índice de la puerta con el anagrama “regalo”.

Se asegura que existe una única puerta que contenga el anagrama “regalo”. Los índices de las puertas comienzan en 0

³Un anagrama es una palabra que resulta de la transposición de todas las letras de otra palabra. Por ejemplo, “grelao” es un anagrama de “regalo”, “grellt” no es un anagrama de “regalo”

Ejemplos

Entrada 1	Salida 1
10 1 9 8 4 14 3 3 21 3 12 HVSfJM URadpn OQIZWt svkMpe FcwIzU jUHodr OjLXUr mDgbPz jUODr1 XsUqGd	5

Nota

En el ejemplo, la palabra oculta en cada puerta es:

- Puerta 0: "GURelL"
- Puerta 1: "LlrugE"
- Puerta 2: "GIAROl"
- Puerta 3: "orglla"
- Puerta 4: "RoiUlG"
- Puerta 5: "gRElao"
- Puerta 6: "LgIUro"
- Puerta 7: "rllgUe"
- Puerta 8: "gRLAoi"
- Puerta 9: "LgIeUr"

De todas las puertas, la puerta cinco "jUHodr" tiene un valor de rotación igual a 3 y luego de decodificar se obtiene la palabra "gRElao", la cual ordenadamente e ignorando mayúsculas es "regalo", es decir, contiene las letras necesarias para formar la palabra "regalo".

I. Infección

Límite de tiempo: 2 segundos

Límite de memoria: 256 megabytes

Doctor Mario descubrió tres nuevas especies de virus: los virus azules, los amarillos y los rojos. Estos se reproducen entre sí de una forma bastante particular:

Sean, r_i, b_i, y_i las cantidades de virus rojos, azules y amarillos respectivamente durante el segundo i . Entonces, en el segundo $i + 1$:

$$r_{i+1} = r_i + b_i$$

$$b_{i+1} = b_i + y_i$$

$$y_{i+1} = y_i + r_i$$

Mr.⁴ L está trabajando en su tesis doctoral sobre sistemas dinámicos discretos no acotados. Desea determinar, dadas las cantidades iniciales r_0, b_0, y_0 , si existe algún segundo $i < 67^{42^{998244353}}$ en el que dos especies tengan la misma cantidad de virus, es decir, si se cumple alguna de las siguientes igualdades:

- $r_i = b_i$
- $b_i = y_i$
- $r_i = y_i$

Por favor ayuda a Luigi con su investigación encontrando la respuesta para cada caso de prueba para que pueda convertirse en Doctor Luigi.

Entrada

La primera línea contiene un entero t , el número de casos de prueba $1 \leq t \leq 10^5$.

Cada caso contiene una única línea con 3 enteros r_0, b_0, y_0 , que representan las cantidades iniciales de virus rojos, amarillos y azules respectivamente ($1 \leq r_0, y_0, b_0 \leq 10^{18}$).

Salida

Para cada caso de prueba, imprime SI si existe algún entero $i < 67^{42^{998244353}}$ tal que $r_i = y_i$, $y_i = b_i$ o bien $r_i = b_i$. En caso contrario imprime NO.

La respuesta no diferencia entre mayúsculas y minúsculas.

Ejemplos

Entrada 1	Salida 1
2	NO
1 2 3	SI
1 2 2	

⁴«Mr.» es una abreviación de magíster.

J. Jamboree de minijuegos

Límite de tiempo: 2 segundos

Límite de memoria: 256 megabytes

Es noche de fiesta en el Reino Champiñón y, como siempre, Mario y Luigi discuten quién es el mejor en los minijuegos. Para resolverlo, Mario cayó en un Espacio de Duelo y desafió a su hermano a un minijuego especial: *Bloques en la esquina*.

Lo que ninguno sabía es que Bowser escondió bajo el tablero el Minimizador, alimentado con uno de los Cristales del Cielo que les robó, capaz de encoger los bloques golpeados hasta hacerlos desaparecer.

El minijuego se juega sobre un tablero cuadrado de $n \times n$ bloques. El bloque en la fila i y columna j contiene $a_{i,j}$ monedas. Algunos bloques fueron saboteados por Bowser: en ese caso $a_{i,j}$ es negativo y, al golpearlo, el jugador pierde $|a_{i,j}|$ monedas.

Mario y Luigi juegan por turnos, comenzando por Mario. En su turno, el jugador elige una de las siguientes dos esquinas del tablero actual:

- Superior izquierda: golpeando todos los bloques de la fila superior y de la columna izquierda.
- Inferior derecha: golpeando todos los bloques de la fila inferior y de la columna derecha.

El jugador suma las monedas de todos los bloques que golpeó (el bloque de la esquina se cuenta una sola vez). Luego, el Minimizador hace desaparecer esos bloques, por lo que el tablero de tamaño $k \times k$ se reduce a uno de $(k - 1) \times (k - 1)$. El minijuego termina cuando no quedan bloques.

Ambos hermanos juegan de forma óptima, buscando maximizar las monedas que obtiene cada uno.

Determina cuántas monedas tendrá Mario al terminar el minijuego.

Entrada

La primera línea contiene un entero n , el tamaño del tablero. ($1 \leq n \leq 2000$)

Las siguientes n líneas contienen n enteros cada una, separados por espacios. El j -ésimo entero de la i -ésima línea es $a_{i,j}$, la cantidad de monedas del bloque ubicado en la fila i y la columna j . ($-10^9 \leq a_{i,j} \leq 10^9$)

Salida

Imprime un único entero: la cantidad de monedas que tendrá Mario al terminar el minijuego, si ambos jugadores juegan de forma óptima.

Ejemplos

Entrada 1	Salida 1
3 7 7 3 3 7 4 9 9 3	35
Entrada 2	Salida 2
3 -2 5 -1 4 -3 6 -5 1 2	0

Entrada 3	Salida 3
7 -20 27 -12 27 18 7 48 -1 11 -4 -7 21 42 -7 -11 28 13 18 -5 24 40 43 -5 46 11 25 38 24 -10 -12 -9 43 19 13 8 1 21 35 44 22 21 43 31 20 2 5 39 8 24	453

Nota

En el primer ejemplo, la esquina superior izquierda le da a Mario más monedas, $7 + 7 + 3 + 3 + 9 = 29$ (Figura 5), pero Luigi responde con la misma y obtiene $7 + 4 + 9 = 20$, dejándole un bloque con 3. Mario terminaría con 32.

7	7	3
3	7	4
9	9	3

Figura 5: Bloques golpeados por Mario (en amarillo) al elegir la esquina superior izquierda.

Es mejor la esquina inferior derecha, con $9 + 9 + 3 + 4 + 3 = 28$ monedas (Figura 6). Luigi obtiene 17 con cualquiera y a Mario le queda un bloque con 7, terminando con 35.

7	7	3
3	7	4
9	9	3

Figura 6: Bloques golpeados por Mario (en amarillo) al elegir la esquina inferior derecha.

En el segundo ejemplo, con la esquina superior izquierda Mario obtiene 1, Luigi toma 9 y a Mario le queda un bloque con -3 , terminando con -2 . Con la inferior derecha obtiene 3, Luigi toma 7 y a Mario le queda -3 , terminando con 0.

K. Kamek y su eterno rival

Límite de tiempo: 2 segundos

Límite de memoria: 256 megabytes

Desde que Yoshi arruinó su plan de secuestrar al Bebé Mario, Kamek busca revancha contra su eterno rival. Esta vez, el Magikoopa cree haber encontrado su punto débil: nadie en la Isla de Yoshi sucumbe ante la fruta como Yoshi.

Kamek preparó un camino con x pedestales en fila. En su bodega guarda n manzanas y m melones, cada uno con un hechizo distinto, por lo que no hay dos frutas iguales.

Para que Yoshi no sospeche, Kamek quiere poner exactamente a manzanas y b melones sobre los pedestales, como máximo una fruta por pedestal, dejando vacíos los demás. Dos trampas son distintas si algún pedestal tiene una fruta en una y , en la otra, una fruta distinta o está vacío.

Determina cuántas trampas distintas puede armar Kamek. Como la respuesta puede ser muy grande, imprímela módulo $10^9 + 7$.

Entrada

La única línea contiene cinco enteros x, n, m, a, b , separados por espacios: x , el número de pedestales; n , el número de manzanas; m , el número de melones; a , las manzanas que quiere poner Kamek; y b , los melones que quiere poner. ($1 \leq x, n, m \leq 10^6, 0 \leq a \leq n, 0 \leq b \leq m, a + b \leq x$)

Salida

Imprime un único entero: la cantidad de trampas distintas, módulo $10^9 + 7$.

Ejemplos

Entrada 1	Salida 1
3 1 1 1 1	6
Entrada 2	Salida 2
4 3 2 2 1	144
Entrada 3	Salida 3
1000000 1000000 1000000 300000 400000	993029403

Nota

En el primer caso de ejemplo, Kamek debe poner su única manzana y su único melón en 3 pedestales. Las 6 trampas posibles se muestran en la Figura 7.

Trampa	Pedestal 1	Pedestal 2	Pedestal 3
1	Manzana	Melón	—
2	Melón	Manzana	—
3	Manzana	—	Melón
4	Melón	—	Manzana
5	—	Manzana	Melón
6	—	Melón	Manzana

Figura 7: Las 6 trampas que puede armar Kamek en el primer caso de ejemplo.

L. Loops Goop

Límite de tiempo: 1 segundo

Límite de memoria: 128 megabytes

El sol de Isla Delfino funciona con Shine Sprites: las estrellitas cuya luz mantiene al balneario brillante y cálido. Por eso, cuando Bowser Jr., disfrazado de Shadow Mario, se dedicó a vandalizar la isla con su Pincel Mágico, todo se vino abajo: embadurnó m trazos de *goop* mágico por las pasarelas y tuberías que unen sus n puntos, los Shine Sprites huyeron del desastre, y la Plaza Delfino se apagó.

Así que Mario toma a FLUDD, una mochila-bomba de agua, y sale a dejar la isla reluciente, punto por punto. El goop viene en muchos **colores**, y el goop de un mismo color forma un sendero: una sucesión de trazos de un punto a otro que nunca pasa dos veces por el mismo trazo. Un chorro de agua de FLUDD lava un sendero entero sin problema, así que un color solo nunca es un dolor de cabeza.

Las mezclas son otra historia. Donde el goop de **dos colores** se enlaza formando un **circuito cerrado**, los dos colores reaccionan entre sí, y la reacción hace imposible limpiarlo. Solo la luz de un **Shine Sprite** logra detener la reacción del circuito cerrado. Por suerte la reacción es delicada: si un tercer color es parte del circuito, envenena la reacción y la apaga. Una mancha realmente terca es siempre obra de exactamente dos colores.

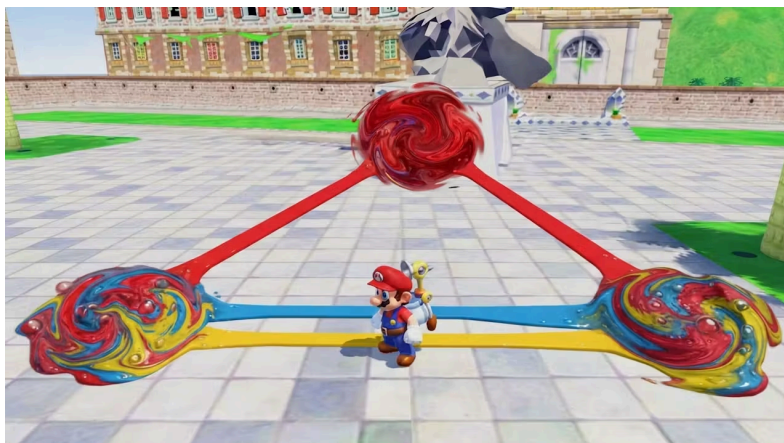


Figura 8: Aquí se ven tres circuitos cerrados; ROJO-AZUL, ROJO-AMARILLO y AZUL-AMARILLO. Se necesitan 3 **Shine Sprite** para detener las reacciones.

Antes de que Mario se agote, el Consejo Pianta quiere un informe: cuántos de esos remolinos de dos colores acechan en la isla, para saber cuántos Shine Sprites reunir. Formalmente, un par de colores (e, f) es **peligroso** si, usando únicamente los trazos de esos dos colores, se puede recorrer un circuito cerrado. Ayuda a Mario a contar **cuántos pares de colores son peligrosos**.

Entrada

La primera línea contiene dos enteros n y m : la cantidad de puntos y de trazos de goop ($0 \leq n \leq 10^4$, $0 \leq m \leq 10^4$).

Cada una de las siguientes m líneas contiene tres enteros u_i , v_i y c_i : un trazo de color c_i entre los puntos u_i y v_i ($1 \leq u_i, v_i \leq n$, $u_i \neq v_i$, $1 \leq c_i \leq m$).

Dos trazos distintos pueden unir el mismo par de puntos, siempre que sean de colores diferentes. Los trazos de un mismo color forman un camino simple.

Salida

Imprime un único entero: la cantidad de pares no ordenados de colores (e, f) con $e \neq f$ que son peligrosos.

Ejemplos

Entrada 1	Salida 1
4 5 1 2 1 2 3 1 3 4 2 4 1 2 1 3 3	3

Entrada 2	Salida 2
5 4 1 2 1 2 3 1 3 4 2 4 5 2	0

M. Mansión de Luigi 4

Límite de tiempo: 0.5 segundos

Límite de memoria: 256 megabytes

Luigi ha recibido una misteriosa carta que, coincidentemente, abrió el **6 de junio de 2026**, a las **06:06:06.666666666** del día.

Obbobobo Boobo,

Boobobob obbobbobbo boo o bbobób bo bo obóbobo bobbobobobób ob ob BOO BBOBB BBOB BBOBOB, bo bobobo obo bobboób obobbobo bo 2000 bobbob boobbobob bobobobbob b 20000 bobbob boobbobob ob bobob.

Ob ob bobob bboboo ob bo bobboboo bo obbo bobbobbo; boobo, bob bobbob bob obbob 4 bobob boo bo bo bobobo ¡Bobo bo bobobob boo obbo bob bo ob obo bbobbo bobbob!

Bo bobboób boobbo bob bobo bo boboboboo, bobbo obo boobbo bobbo obo bobo bo bobo bobo boo boo bob bobóbobob bo bobbob bobobobob. Bo obbobobob obbobobbo.

OBBB bobbobobobbob, obbobbo boo bo bobob bobbobo boobbo bobobbo obbo bobbo.

Obbo,

Bob Boo

Luigi no puede leer Boogio, pero sabe bien de qué trata esto.

—¡Debo haber ganado nuevamente una mansión!

Y como ya lo han engañado tres veces con mansiones, es imposible que la cuarta también sea un engaño. Luigi tomó la decisión de aceptar el premio e inmediatamente viajó a la mansión.

Una vez allá, ya dentro, el Rey Boo le dijo «Ajaajajajaj era una trampa, jamás podrás salir de esta mansión» y luego desapareció sin dejar rastro. Luigi intentó salir pero no pudo.

Dentro de la habitación se encuentra un mapa de la mansión y Luigi, rápidamente, se dio cuenta de que solo existe una forma de ir desde cualquier habitación a otra, y de que es posible ir a cualquier habitación desde la habitación en la que se encuentra.

La mansión tiene n habitaciones y cada habitación tiene una lámpara. El Rey Boo dejó unas encendidas y otras apagadas. Para no pasar la noche a oscuras entre fantasmas, Luigi quiere dejarlas todas encendidas usando la menor cantidad de movimientos posible.

Pero Luigi es un miedoso terrible: cada vez que entra corriendo a una habitación, del susto golpea el interruptor y la lámpara de esa habitación se invierte: una encendida se apaga y una apagada se enciende. Parado en una habitación, Luigi puede hacer una de estas dos cosas, y cada una cuenta como un movimiento:

- correr a una habitación vecina y, al entrar corriendo, invierte la lámpara de *esa* habitación;
- calmarse e invertir la lámpara de la habitación donde está.

Antes de que Luigi empiece a moverse, de alguna parte de la habitación en la que se encuentra puede escuchar al profesor E. Gadd. «Luigi, todo lo que puedo hacer para ayudarte es teletransportarte a tu habitación preferida una única vez, aparecerás ahí sin que la lámpara de esa habitación cambie».

Luigi debe decidir dónde empezar y no importa dónde termine. ¿Cuál es el mínimo número de movimientos para dejar todas las lámparas encendidas?

Entrada

La primera línea contiene un entero n : la cantidad de habitaciones ($1 \leq n \leq 2 \cdot 10^5$).

La segunda línea contiene n enteros $c_1, \dots, c_n$, donde $c_i = 1$ si la lámpara de la habitación i está encendida y $c_i = 0$ si está apagada.

Cada una de las siguientes $n - 1$ líneas contiene dos enteros u_i y v_i ($1 \leq u_i, v_i \leq n$, $u_i \neq v_i$): un pasillo que une las habitaciones u_i y v_i . Se asegura que se puede visitar todas las habitaciones desde la habitación 1, y que hay un único camino desde una habitación a otra.

Salida

Imprime un único entero: el mínimo número de movimientos para encender todas las lámparas.

Ejemplos

Entrada 1	Salida 1
3 0 1 0 1 2 2 3	4
Entrada 2	Salida 2
5 1 0 0 0 0 1 2 1 3 1 4 1 5	8

N. Negociación Poliedral

Límite de tiempo: 1 segundo

Límite de memoria: 128 megabytes

Mario y Wario se encontraron por sorpresa en la torre principal del castillo del mundo 8 tras vencer al malvado Bowser. Dentro de uno de sus baúles secretos se encontraba una gran sorpresa: n poliedros con valores que podrían volver millonario a cualquiera dentro del Reino Champiñón!

La magia de estos poliedros yacía en que permitían a su portador adquirir cualquier poder: el de la flor de fuego, convertirse en tanuki, estrella, entre muchos otros, por lo que eran objetos muy buscados por los aventureros del reino.

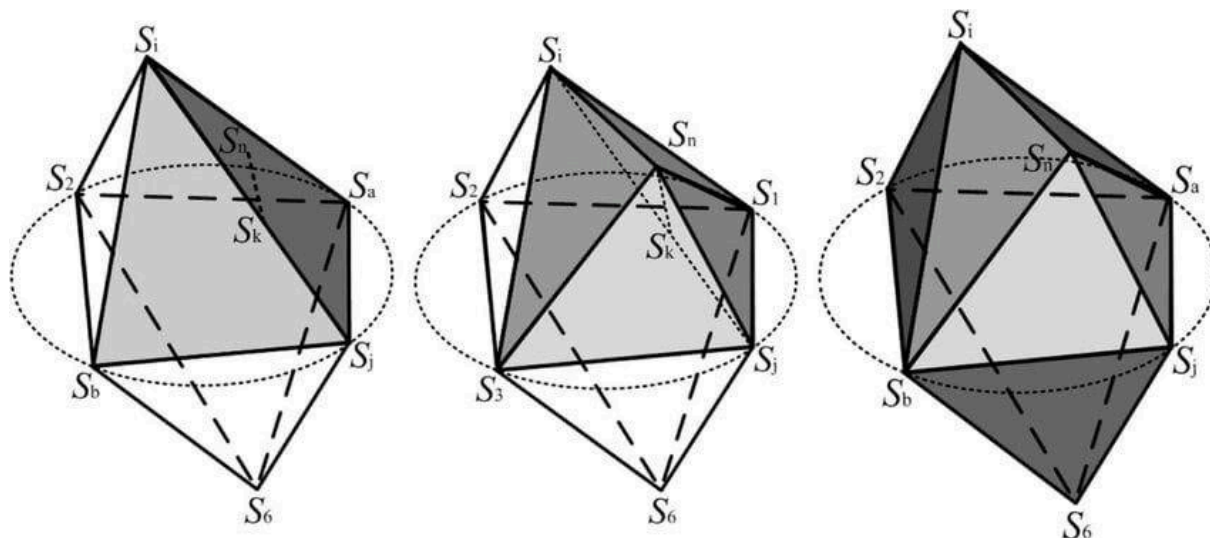


Figura 9: Ilustración encontrada en uno de los libros de magia oscura de la biblioteca de bowser, mostrando cómo activar el poder de uno de estos poliedros.

Al fin y al cabo, la razón por la que los poliedros eran tan valiosos, además de sus poderes, era que cada uno se vendía por una cantidad enorme de monedas. Los poliedros del baúl están enumerados desde 0 hasta $n - 1$, y el i -ésimo de ellos tiene un valor de 2^i monedas.

Como Mario y Wario se encontraron al mismo tiempo en la habitación del baúl, su primer instinto fue batirse en duelo para determinar quién se quedaría con el tesoro. Sin embargo, ambos estaban demasiado cansados de su larga aventura, por lo que se sentaron a hacer una lluvia de ideas.

Una de las ideas que ganó fue dividirse los poliedros de la forma más justa posible; es decir, de modo que la diferencia absoluta entre lo que ganaría cada uno vendiendo todos sus poliedros fuese la mínima posible.

Si Mario y Wario hubiesen tenido suficiente energía como para encontrar esta repartición justa, la habrían gastado enfrentándose en un uno contra uno en lugar de negociar. Esto significa que será tu trabajo determinar cuál será la mínima diferencia absoluta que pueden alcanzar.

Entrada

La primera línea contiene un entero n ($1 \leq n \leq 10^6$), el número de poliedros en el baúl.

Salida

Imprime un único entero: la mínima diferencia absoluta que pueden alcanzar Mario y Wario al dividir los poliedros en dos grupos.

Ejemplos

Entrada 1	Salida 1
3	1

Nota

En el ejemplo, los poliedros valen 1, 2 y 4 monedas. Si uno se queda con el de 4 y el otro con los de 1 y 2, la diferencia es $4 - 3 = 1$, y no es posible lograr una diferencia menor.